Penalty Football

Álvaro Martín Losa
1º DAW
Repositorio

1. DESCRIPCIÓN GENERAL DEL PROYECTO

1 .1 Objetivo del Juego

Penalty Football es una aplicación interactiva desarrollada en el lenguaje de programación **Java** para ejecutarse a través de la consola de comandos. El objetivo principal del usuario es asumir el rol de un tirador de penaltis en una tanda de 5 lanzamientos por ronda, acumulando la mayor cantidad de goles y recompensas económicas virtuales mientras gestiona estratégicamente sus puntos de energía acumulados y accede a los servicios de la tienda del juego.

1 .2 Reglas del Juego y Mecánicas

- **Tanda de Penaltis:** Cada partida se compone de 5 lanzamientos obligatorios consecutivos.
- **Opciones de Tiro:** El usuario puede elegir disparar a tres zonas específicas de la portería: **izquierda, centro o derecha.**
- **Inteligencia Aleatoria del Portero:** En cada tiro, el portero de forma autónoma escoge una dirección de parada mediante un generador de números pseudoaleatorios.
- **Resolución del Tiro:**
  - Si la posición elegida por el jugador coincide exactamente con la del portero, el tiro es parado(**Parada**).
  - Si la posición del jugador es distinta a la del portero, la jugada termina en **Gol**.
- **Gestión de Energía:** Cada ronda de lanzamientos consume un coste fijo de **10 puntos de energía**. El jugador inicia con un máximo de 100 puntos y debe recuperar energía mediante la opción de descanso cuando su nivel sea menor o igual a 10.
- **Economía del Juego:** Cada gol anotado otorga **100 €**. Si el jugador consigue anotar los 5 lanzamientos de la tanda (pleno), recibe un **Bonus de Partido Perfecto de 500 €** adicionales.

1 .3 Desarrollo de una Partida (Flujo Principal)

Al ejecutar la aplicación, el usuario interactúa con un menú principal interactivo numérico (1 a 5). El ciclo de juego se desarrolla de la siguiente manera:

1. El programa muestra el título estilizado mediante secuencias de escape **ANSI** y despliega el menú de navegación.
2. Al seleccionar la **Opción 1 (Iniciar Juego)**, se verifica que la energía sea superior a 10. De cumplirse, se descuentan 10 unidades de energía y se reinician los contadores locales de la ronda.

3. Se ejecuta un bucle **for** de 5 iteraciones. En cada iteración, se solicita la posición del tiro. El sistema valida mediante un bucle **do - while** que la entrada escrita sea válida (ignorando mayúsculas y espacios innecesarios).
4. Se genera el movimiento del portero y se evalúa el resultado, mostrando avisos coloreados en consola (Verde para Gol, Rojo para Parada).
5. Al finalizar los 5 tiros, se calcula el dinero obtenido en la ronda, se suma al saldo total acumulado y se muestra un desglose de las estadísticas de la tabla

1 .4 Condiciones de Victoria, Derrota y Finalización

- **Victoria de Ronda / Partido Perfecto:** Se considera una victoria destacada cuando se meten 5 goles de 5 posibles disparos, desbloqueando la bonificación económica máxima.
- **Derrota / Bloqueo por Agotamiento:** Si el nivel de energía disminuye a 10 o menos puntos, el sistema impide iniciar nuevas partidas, forzando al usuario a elegir la opción de descansar para recuperarse.
- **Finalización del Programa:** El juego concluye cuando el usuario selecciona explícitamente la **Opción 5 (Salir)** en el menú principal, mostrándose un mensaje final con el total del dinero acumulado a lo largo de la sesión.

2. FUNCIONALIDADES IMPLEMENTADAS Y ANÁLISIS COMPARATIVO

2.1 Funcionalidades Implementadas

El programa 'JuegoFutbol.java' integra de manera sólida las siguientes características funcionales:

- **Menú de Consola Interactivo:** Navegación robusta con 5 opciones diferenciadas gestionadas por una estructura `switch`.
- **Validación Estricta de Entradas:** Tratamiento de cadenas mediante **. toLowerCase(). trim()** para asegurar tolerancia a mayúsculas y espacios flotantes.
- **Motor de Juego Aleatorio:** Selección aleatoria de posiciones del portero mediante la clase **java.util.Random** y vectores.
- **Sistema de descanso y recuperación:** Recuperación progresiva de 10 puntos de energía por sesión de descanso hasta alcanzar el tope máximo de 100 puntos.
- **Histórico y Estadísticas:** Acumulación continua de tiros a la izquierda, centro, derecha, goles, paradas, dinero en ronda y dinero global.
- **Tienda de consumibles:** Adquisición interactiva de pociones de tiro y energía con confirmación repetitiva.
- **Estilización ANSI:** Uso de códigos de colores en consola para enfatizar encabezados, goles, fallos y mensajes de error.

2.2 Tabla Comparativa: Propuesta Inicial (README.md) vs. Código Real (.java)

Característica	Propuesta Inicial (README.md)	Implementación Real (JuegoFutbol.java)	Estado / Diferencias
Lanzamientos por Ronda	5 tiros por tanda frente a portero aleatorio.	Implementado mediante bucle `for ( contador = 0; contador < 5; contador++)`.	**100% CONFORME**
Sistema de Energía	Consumo de 10 puntos por juego; descanso regenera 10 puntos.	Restricción de jugar con ≤10 de energía. Recuperación sumando 10 hasta un tope de 100.	**100% CONFORME**
Control de Acceso (Regla del Primer Juego)	Las opciones 2, 3 y 4 requieren haber jugado al menos 1 ronda.	Las opciones 3 (Estadísticas) y 4 (Tienda) verifican `partidasJugadas > 0`. La opción 2 (Descansar) permite ejecutarse libremente.	**DIVERGENCIA MENOR** (Descanso accesible en inicio)
Sistema de Tienda y Pociones	Permite comprar pociones de tiro y energía.	Permite seleccionar y comprar incrementando `pocionTiro++` y `pocionEnergia++`. Se valida con bucle "s/n".	**FUNCIONALIDAD BASE**
Uso Transaccional de Pociones y Dinero	Implícito en la economía del juego.	Las pociones no tienen efecto activo en la jugabilidad.	**PENDIENTE 2ª EVAL.** (Diseñado como contador)

3. ARQUITECTURA Y DISEÑO TÉCNICO DEL SOFTWARE

3.1 Diccionario de Variables

Variable	Tipo de Dato	Ámbitos de Uso	Propósito y Descripción
`preguntaTirar`	`String`	Local al `main` / Bucle tiros	Almacena la cadena enviada por el jugador sobre dónde quiere disparar.
`preguntaTirarMinus`	`String`	Local al `main` / Bucle tiros	Guarda la versión normalizada en minúsculas y sin espacios de la elección del tirador.
`posicionTiro`	`String`	Local al `main` / Bucle tiros	Almacena el resultado aleatorio del tiro del portero ("izquierda", "derecha", "centro").
`posicionValida`	`boolean`	Local al `main` / Bucles `do- while`	Flag de control para comprobar si la entrada del usuario es una dirección válida.
`posiciones`	`String[]`	Local al `main` (Array)	Vector con los 3 valores posibles donde se puede lanzar o estirar el portero.
`opcionMenu`	`int`	Local al `main` / `switch`	Guarda la opción seleccionada por el usuario en el menú principal.
`partidasJugadas`	`int`	Global al `main`	Contador acumulativo de partidas jugadas para desbloquear opciones.
`dineroTotal` / `dineroRonda`	`int`	Global / Local a la tanda	Acumula los ingresos por goles anotados y bonificaciones acumuladas.
`goles` / `paradas`	`int`	Local a la tanda	Contadores de aciertos y fallos en la tanda en curso.
`tiroIzquierda` / `tiroDerecha` / `tiroCentro`	`int`	Local a la tanda	Contadores estadísticos de la distribución de dirección de los tiros.
`energía`	`int`	Global al `main`	Puntos de energía del jugador (inicia en 1 00, baja -1 0 por juego, sube +1 0 al descansar).
`pocionTiro` / `pocionEnergia`	`int`	Global al `main`	Contadores del inventario de pociones compradas en la tienda.

3.2 Tabla de Constantes del Sistema

Constante	Tipo	Valor Asignado	Utilidad Técnica
`PREMIO_GOL`	`final int`	`1 00`	Recompensa en euros otorgada por cada gol anotado individualmente.
`BONUS_PERFECTO`	`final int`	`500`	Bonificación extraordinaria si se marcan los 5 penaltis en una misma tanda.
`TIROS`	`final int`	`5`	Establece el número exacto de lanzamientos por ronda de juego.
`RESTABLECER`, `CIAN`, `VERDE`, `AMARILLO`, `ROJO`	`final String`	Códigos de escape Unicode (`\u001 B[. . . ]`)	Estilización de fuentes e interfaces de la consola Java.

3.3 Estructuras de Control Utilizadas

La aplicación demuestra un dominio de las tres estructuras fundamentales de control de flujo:

- **Bucle Principal** `do-while (opcionMenu ! = 5)`**:** Mantiene la ejecución ininterrumpida de la aplicación hasta que el usuario decida salir explícitamente.
- **Estructura Selectiva** `switch (opcionMenu)`**:** Canaliza la ejecución del programa hacia la funcionalidad correspondiente (Jugar, Descansar, Estadísticas, Tienda, Salir) o captura selecciones inválidas en su rama `default`.
- **Bucle Determinado** `for (int contador = 0; contador < TIROS; contador++)`**:** Gobierna
  la secuencia de los 5 lanzamientos dentro del caso 1 .
- **Bucles de Validaciones** `do-while (! posicionValida)` **y** `do-while (! compraValida)`**:** Atrapan al usuario en una secuencia de reintento hasta que introduzca valores comprensibles por el parser.
- **Estructuras Condicionales** `if-else if-else`**:** Evalúan las colisiones de tiro contra el movimiento del portero, aplican la lógica de premios y controlan los topes de energía.

3.4 Explicación Detallada del Uso de Arrays e Inteligencia del Portero

El sistema utiliza un arreglo unidimensional de cadenas de caracteres (`String[] `) inicializado con tres posiciones estáticas:

```
String[] posiciones = {" izquierda", " derecha", " centro"};
```

Para determinar la parada o tirada del portero, se emplea la clase **java.util.Random**. Se obtiene un índice numérico entero mediante **aleatorio. nextInt (posiciones. length)**, el cual devuelve de forma pseudoaleatoria un valor en el rango `[0, 2]` .

Dicho valor numérico se utiliza como índice sobre el array `posiciones` para obtener dinámicamente la cadena del movimiento del portero:

```
posicionTiro = posiciones[ aleatorio. nextInt( posiciones. length) ] ;
```

Este diseño permite una extensión limpia para incluir nuevas zonas (**"escuadra_izquierda"** o **"panenka"**) modificando únicamente el contenido del array sin necesidad de reescribir la lógica del motor aleatorio.

4. PLAN DE PRUEBAS Y VALIDACIÓN DEL SISTEMA

Se ha llevado a cabo una batería intensiva de pruebas funcionales dividida en tres categorías: casos nominales, casos incorrectos y casos límite.

4.1 Casos de Prueba Normales (Nominales)

ID	Descripción	Entrada Proporcionada	Resultado Esperado	Resultado Obtenido	Estado
CN-01	Inicio de juego con energía completa (1 00)	Seleccionar menú 1	Permite jugar, resta 1 0 de energía (queda en 90) e inicia la tanda.	Juego iniciado correctament e, energía en 90.	PASA
CN-02	Realizar tiradas válidas en tanda	`izquierda`, `centro`, `derecha`	Acepta cada tiro, calcula parada o gol y suma estadísticas.	Tiros procesados y contadores incrementados.	PASA
CN-03	Descansar para ganar energía	Seleccionar menú 2 (Energía = 80)	Aumenta la energía a 90 y muestra mensaje descriptivo.	Energía actualizada a 90.	PASA
CN-04	Consulta de estadísticas tras jugar	Seleccionar menú 3 (partidasJugadas > 0)	Despliega tabla detallada con dinero, energía y pociones.	Estadísticas mostradas correctamente en pantalla.	PASA

4.2 Casos de Prueba Incorrectos (Manejo de Errores de Usuario)

ID	Descripción	Entrada Proporcionada	Resultado Esperado	Resultado Obtenido	Estado
CI-01	Opción inválida en Menú Principal	`9` o `-1`	Muestra mensaje en rojo y vuelve a pedir menú.	Aviso de opción inválida y bucle mantenido.	PASA
CI-02	Posición de tiro errónea o tipográfica	`arriba` o `izqda`	Muestra "Posicion incorrecta" y exige reintento.	Rechaza la entrada y repite el prompt sin gastar tiro.	PASA
CI-03	Ver estadísticas sin haber jugado	Menú `3` con `partidasJugadas = 0`	Informa de que no hay datos registrados para mostrar.	Mensaje de advertencia emitido correctamente.	PASA
CI-04	Respuesta no válida al continuar en Tienda	`quizas` (en vez de 's' o 'n')	Muestra "Opcion no válida, responde s o n".	Bucle de confirmación retiene el flujo hasta dar 's' o 'n'.	PASA

4.3 Casos de Prueba Límite y Condiciones de Envolvente

ID	Descripción	Entrada Proporcionada	Resultado Esperado	Resultado Obtenido	Estado
CL-01	Intento de jugar con energía insustancial (≤ 10)	Menú `1` con `energia = 10`	Deniega la partida y sugiere ir a descansar al menú.	Bloqueo exitoso de partida y sugerencia de descanso.	PASA
CL-02	Intento de descansar con energía al máximo (100)	Menú `2` con `energia = 100`	Informa de que la energía está al tope y no suma más.	No sobrepasa el límite de 100 puntos.	PASA
CL-03	Pleno de 5 goles en una misma tanda	5 goles consecutivos acertados	Suma 500€ de goles + 500€ de BONUS_PERFECTO (1 000€ total).	Bonus disparado correctamente con mensaje amarillo.	PASA
CL-04	Tolerancia a mayúsculas y espacios	`" IZQUIERDA "`	Normaliza a `"izquierda"` y lo acepta como tiro válido.	`.toLowerCase().trim()` procesa la cadena sin error.	PASA

5. DIFICULTADES ENCONTRADAS, SOLUCIONES Y AMPLIACIONES

5.1 Dificultades Técnicas y Soluciones Aplicadas

1. **Manejo de Buffer y Caracteres en Blanco:**
   Problema: Al leer la entrada del usuario mediante `BufferedReader`, pequeños descuidos al teclear espacios adicionales o letras mayúsculas provocaban fallos continuos de validación en la comparación estricta de `String`.
   Solución: Se aplicó la concatenación de métodos `. trim(). toLowerCase()` sobre la cadena leída antes de realizar la comparación lógica con los datos esperados.

2. **Generación de Rangos Válidos para Arreglos:**
   Problema: Riesgo de desbordamiento de índice **(ArrayIndexOutOfBoundsException)** al mapear el número aleatorio con las posiciones del portero.
   Solución: Se indexó el generador directamente sobre la propiedad dinámica del array **posiciones. length**, garantizando que el límite superior siempre se ajuste al tamaño del vector.

3. **Gestión de Persistencia de Estados entre Ciclos:**
   Problema: Reinicio no deseado de contadores al volver al menú principal.
   Solución: Se declararon las variables globales acumulativas **(dineroTotal, energia, partidasJugadas)** fuera del cuerpo del bucle **do- while** principal, reservando la reinicialización interna solo para los contadores específicos de la tanda.

5.2 Propuesta de Ampliaciones para la Segunda Evaluación

De cara al segundo trimestre, se proyectan las siguientes evoluciones estructurales sobre la base de código actual:

- **Refactorización a Programación Orientada a Objetos (POO):** Descomponer el archivo único **JuegoFutbol. java** en una arquitectura modular de clases:
  - `Jugador`: Gestión de atributos como nombre, energía, dinero e inventario.
  - `Portero`: Más posiciones de posibles paradas
  - `Tienda`: Manejo de productos y transacciones comerciales restando saldo real.
  - `Estadisticas`: Registro histórico detallado exportable.
- **Consumo Funcional de Pociones:** Implementar la utilización real de los elementos comprados en tienda:
  - *Poción de Tiro:* Aumenta la probabilidad de engañar al portero o permite repetir un tiro fallado.
  - *Poción de Energía:* Permite recuperar 50 puntos de energía de forma instantánea sin gastar un turno de descanso.